Biometria - Projekt 1

Katarzyna Skoczylas, Miłosz Zieliński

Marzec 2026

1 Wstęp

Celem projektu było samodzielne opracowanie i implementacja wybranych algorytmów przekształcania obrazów, bez korzystania z gotowych bibliotek czy funkcji. Kluczowym założeniem było dogłębne zrozumienie mechanizmów działania tych operacji oraz stworzenie aplikacji okienkowej umożliwiającej ich praktyczne zastosowanie.

2 Opis aplikacji

Aplikacja została napisana w języku Python. Do zbudowania graficznego interfejsu użytkownika użyliśmy biblioteki *tkinker*, ponieważ jest ona prosta w implementacji, intuicyjna w wykorzystaniu i uznaliśmy ją za wystarczające narzędzie do tego projektu. Ponadto jest dostarczana jako standardowa biblioteka Pythona, więc nie jest wymagana żadna dodatkowa instalacja. Do obsługi obrazów wykorzystalismy bibliotekę *Pillow*, która jest rozwijanym forkiem danwego PIL (*Python Image Library*). W wielu funkcjach wykorzystujemy także standardową bibliotekę *numpy*. Wszystkie potrzebne pakiety znajdują się w pliku `requirements.txt`.

Aplikacja podzielona jest na część logiki i część UI. W folderze logiki znajdują się 3 pliki:

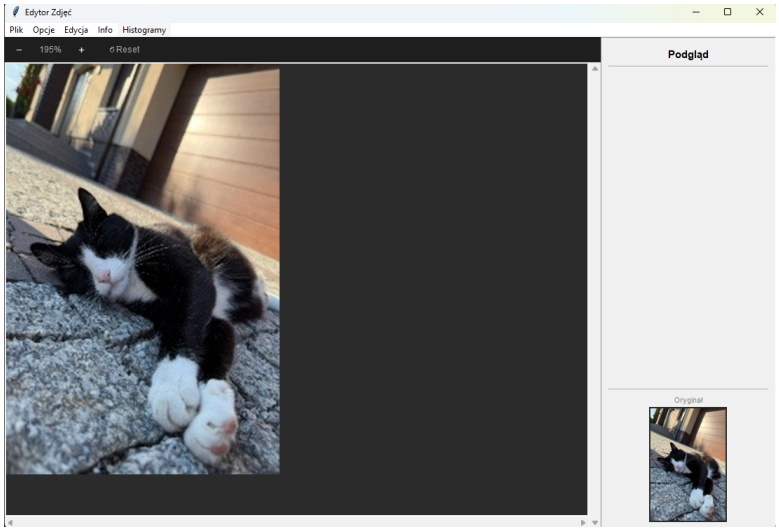
- `file.py` - odpowiada wyłącznie za operacje na plikach: otwieranie, zapisywanie, wybór ścieżki
- `filters.py` - miejsce implementacji wszystkich operacji na obrazie
- `history.py` - realizuje mechanizm cofania zmian (Ctrl+Z) i przechowuje listę 10 ostatnich stanów obrazu.

Część UI składa się z:

- `canvas.py` - centralny obszar roboczy odpowiedzialny za dynamiczne renderowanie przetwarzanego obrazu oraz obsługę skalowania
- `menu.py` - górny pasek sterowania umożliwiający wybór odpowiednich czynności z menu
- `sidebar.py` - boczny panel interaktywny, zawierający kontrolery (suwaki, przyciski) do sterowania parametrami filtrów w czasie rzeczywistym.

`Main_frame.py` to główne okno aplikacji. Łączy wszystkie komponenty UI z logiką.

Główny widok po otwarciu pliku zdjęciowego przedstawia Rysunek 1:



Rysunek 1: Widok główny.

Uwagi techniczne dotyczące implementacji:

- **Obsługiwane formaty plików** to: `.jpg`, `.jpeg`, `.webp`, `.png`, `.bmp`, `.tiff` oraz `.gif`.
- **Zapis obrazu**: aplikacja obsługuje zapis pod tą samą nazwą (Ctrl + S) oraz zapis jako nowy plik (Ctrl + Shift + S).
- **Spójność formatu RGB**: Wszystkie funkcje (nawet te przekształcające obraz na czarno-biały) zwracają wynik w formacie RGB. Ułatwia to nakładanie na siebie wielu filtrów jeden po drugim. W przypadku skali szarości, każdy z trzech kanałów (R, G, B) przechowuje tę samą wartość.
- **Precyzja obliczeń**: Podczas operacji matematycznych wartości pikseli często przekraczają standardowy zakres. Aby uniknąć błędów i utraty danych, obliczenia wykonywane są na liczbach zmiennoprzecinkowych typu `float32`.
- **Normalizacja wyniku (clip)**: Ponieważ końcowy obraz musi być zapisany w standardzie 8-bitowym, stosujemy metodę `np.clip`, która ogranicza wartości do przedziału $[0, 255]$. Na koniec dane są konwertowane na typ `uint8`, zrozumiały dla biblioteki *Pillow*.
- **Obliczenia na brzegach**: Wykorzystaliśmy funkcję `np.pad` w trybie `mode='edge'`. Algorytm powiela wartości skrajnych pikseli, dzięki czemu na brzegach przefiltrowanego obrazu nie powstają czarne ramki, a wynik jest spójny wizualnie.

3 Zaimplementowane filtry

Wszystkie zaimplementowane filtry znajdują się w pliku `filters.py`. Poniżej przedstawione jest zestawienie dostępnych operacji wraz z krótkim opisem matematycznym oraz przykładami ich działania.

3.1 Operacje na pikselach

- **konwersja do odcieni szarości** - do wyboru są 2 algorytmy: prosta średnia z trzech kanałów oraz metoda luminancji (uwzględniająca percepcję ludzkiego oka).
  Dla ludzkiego oka zielony jest około 10 razy jaśniejszy niż niebieski. Na tej podstawie

stworzono wagi dla każdego z kanałów, które lepiej przybliżają ludzką percepcję. Wzór na metodę luminancji prezentuje się następująco:

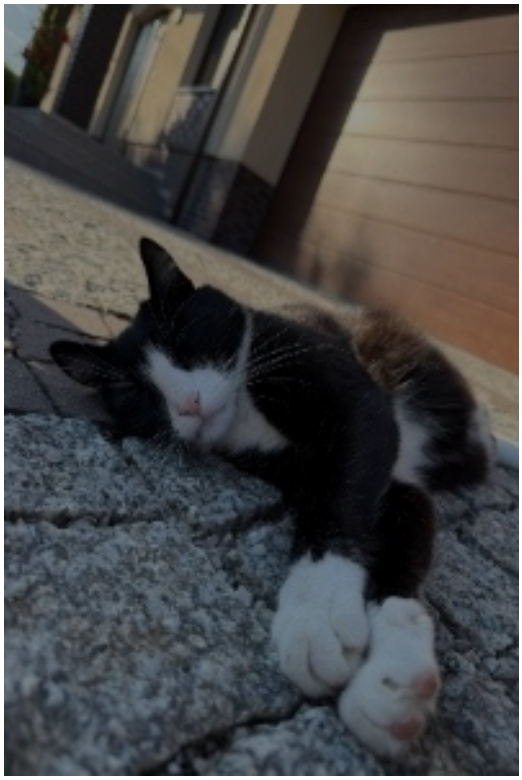
$$J_w = 0.2126R + 0.7152G + 0.0722B,$$

gdzie R, G, B to odpowiednio kanały czerwony, zielony i niebieski.
Na Rysunku 2 oraz 3 przedstawione są wyniki zastosowania odpowiednio uśredniania kanałów i metody luminancji. W tym przypadku różnica jest mało zauważalna.

Rysunek 2: Szarość - uśrednianie kanałów

Rysunek 3: Szarość - metoda luminancji

- **korekta jasności** - ta operacja sprawia, że każdy piksel mnożony jest przez stałą, która jest wybierana przez użytkownika za pomocą suwaka. Na Rysunku 4 oraz 5 widzimy efekt uzyskany przy użyciu różnych parametrów.



Rysunek 4: Jasność ($\alpha = 0.6$)

Rysunek 5: Jasność ($\alpha = 2$).

- **korekta kontrastu** - dla tej operacji posłużyliśmy się następującym wzorem:

$$J_w(x, y) = (J(x, y) - 128) \cdot \alpha + 128,$$

gdzie α to parametr wybierany przez użytkownika. Wartość 128 używana we wzorze reprezentuje środek zakresu obrazu 8-bitowego. Dla $\alpha > 1$ następuje zwiększenie kontrastu poprzez oddalenie wartości jasnych i ciemnych od środka skali. Dla $0 < \alpha < 1$ kontrast ulega zmniejszeniu, zbliżając wszystkie poziomy jasności do neutralnej szarości. Przykład użycia pokazany jest na Rysunku 6 oraz 7

Rysunek 6: Kontrast ($\alpha = 0.6$)

Rysunek 7: Kontrast ($\alpha = 1.9$)

- **negatyw** - to operacja polegająca na odwróceniu skali jasności obrazu. Każdy piksel o wartości J przyjmuje nową wartość $255 - J$. Powoduje to zamianę jasnych obszarów na ciemne i odwrotnie. W obrazach kolorowych aby uzyskać negatyw operację odejmowania przeprowadzamy dla każdej składowej R, G, B. Przykładowy efekt operacji widoczny jest na Rysunku 12

- **binaryzacja** - polega na ustawieniu wartości piksela jako 0 lub 255 w zależności od przyjętego progu zgodnie z wzorem

$$J_w(x, y) = \begin{cases} 255 & \text{dla } J(x, y) > T \\ 0 & \text{dla } J(x, y) \leq T \end{cases}$$

  gdzie T to threshold.
  Ważnym elementem tego algorytmu jest odpowiedni dobór progu odcięcia. W naszej implementacji to użytkownik dobiera go za pomocą suwaka. Operacja działa na jednym kanale, dlatego wewnątrz najpierw wywoływana jest konwersja do odcieni szarości. Obrazy z dwoma różnymi progami odcięcia widać na Rysunku 8 oraz 9

Rysunek 8: Binaryzacja ($T = 60$)

Rysunek 9: Binaryzacja ($T = 128$)

- **potęgowanie** - tą operację zaimplementowaliśmy zgodnie z następującym wzorem z normalizacją i przeskalowaniem do pełnego zakresu:

$$J_w(x, y) = 255 \cdot \left(\frac{J(x, y)}{J_{max}} \right)^{\alpha}$$

  Podnosząc obraz do potęgi zmieniamy poziom jasności i możemy ujawnić niewidoczne wcześniej szczegóły. Dla $\alpha > 1$ dochodzi do przyciemniania obrazu z większym zróżnicowaniem jasnych partii. Dla $\alpha < 1$ uzyskujemy pierwiastkowanie (korektę gamma). W tym przypadku operacja powoduje rozjaśnienie obrazu z większym zróżnicowaniem ciemnych partii. Parametr α wybiera użytkownik. Efekt działania przedstawiają Rysunki 10 oraz 11.

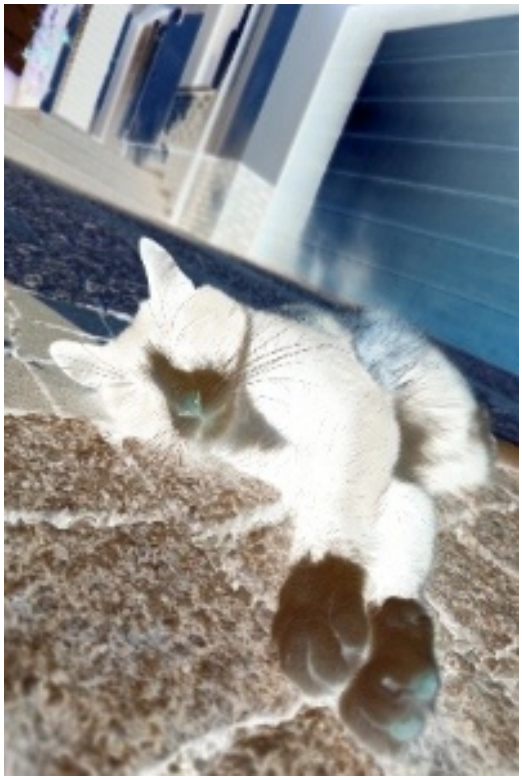
Rysunek 10: Potęgowanie ($\alpha = 0.5$)

Rysunek 11: Potęgowanie ($\alpha = 2$)

- **logarytmowanie** - dla operacji logarytmowania również wykorzystaliśmy wzór z normalizacją i przeskalowaniem do pełnego zakresu:

$$J_w(x, y) = 255 \cdot \frac{\log(1 + J(x, y))}{\log(1 + J_{max})}$$

  Warto zauważyć, że dodanie 1 wynika z tego, że logarytm nie jest określony dla 0, stąd takie przesunięcie. Operacja logarytmowania podobnie jak pierwiastkowania rozjaśnia obraz z większym zróżnicowaniem ciemnych partii. Zastosowanie operacji widoczne jest na Rysunku 13



Rysunek 12: Negatyw.

Rysunek 13: Logarytmowanie.

3.2 Filtry

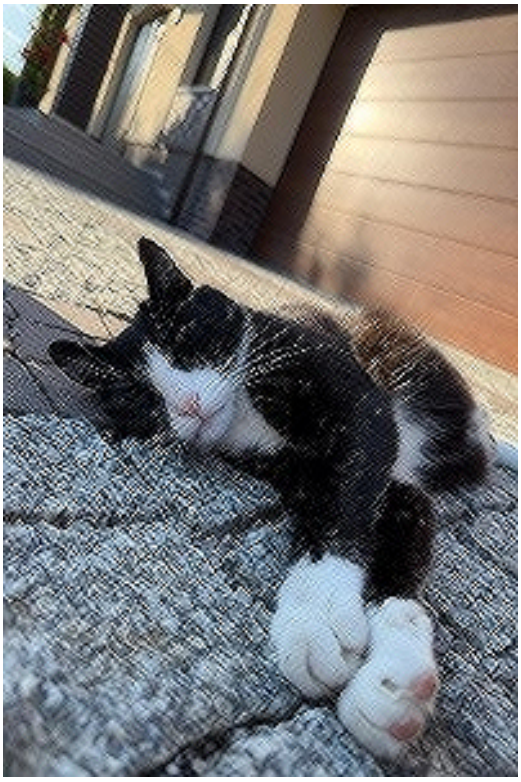
- **filtr uśredniający** - służy do wygładzania obrazu i redukcji szumów. Nowa wartość każdego piksela jest obliczana jako średnia arytmetyczna z jasności pikseli w zadanym oknie sąsiedztwa (do wyboru kernele: 3×3, 5×5, 7×7). Im większy rozmiar okna tym mocniejszy efekt rozmycia.
- **filtr Gaussa** - wykorzystuje dwuwymiarowy rozkład normalny do wagowania pikseli w oknie sąsiedztwa. W przeciwieństwie do zwykłego uśredniania, filtr ten silniej bierze pod uwagę piksele bliższe centrum maski, co pozwala na redukcję szumów przy jednoczesnym lepszym zachowaniu krawędzi obiektów. Użytkownik ma do wyboru 3 kernele: 3×3, 5×5, 7×7. Może on także za pomocą suwaka sterować parametrem σ (określającym szerokość dzwonu), co pozwala na precyzyjną kontrolę stopnia rozmycia obrazu.

Rysunek 14: Filtr uśredniający (kernel: $5 \times 55 \times 5$)

Rysunek 15: Filtr Gaussa (kernel: 5×5)

- **filtr wyostrzający** - wynikiem działania takich fitrów jest wyostrzenie obrazu, a także zwiększenie ilości szumów. Aplikacja oferuje kilka wariantów wyostrzania:
  - Mean Removal - podstawowa wersja filtru górnoprzepustowego, jego użycie powoduje znaczne wyostrzenie obrazu, ale także wzmocnienie wszelkich szumów i zakłóceń
  - HP1, HP2, HP3 - różna charakterystyka wzmocnienia, dzięki różnym kernelom (różne sposoby uwzględniania sąsiednich pikseli)

Rysunek 16: Filtr wyostrzający (Mean Removal)



Rysunek 17: Filtr wyostrzający (HP2)

- **własny filtr** - elastyczne narzędzie, umożliwiające wykonanie splotu z dowolną macierzą zdefiniowaną przez użytkownika i testowanie własnych filtrów. Program pozwala na wybór rozmiaru maski w zakresie od 2×2 do 7×7. Po ustaleniu wymiarów, na ekranie pojawia się macierz odpowiedniego rozmiaru, do której użytkownik może wpisać swoje własne współczynniki.
- **filtry krawędziowe** - wszystkie trzy algorytmy działają poprzez przesuwanie małej macierzy (maski) po obrazie i obliczanie różnic między sąsiednimi pikselami. Wzmacniają one obszary o dużych zmianach intensywności (krawędzie), jednocześnie wygaszając obszary o stałej jasności. W każdym kroku pętli wybierany jest fragment obrazu o takim samym rozmiarze jak maska. Piksele pod maską są mnożone przez odpowiadające im liczby w masce, a wyniki są sumowane.
  - **krzyż Robertsa** - jest to najprostszy operator. Wykorzystuje maski o rozmiarze 2×2, które mierzą gradient wzdłuż kierunków diagonalnych:

    $$K_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad K_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

    Oba kierunki łączone są wzorem na długość wektora. Filtr ten z racji małej maski jest podatny na szumy.
  - **operator Sobela** - ten operator używa masek 3×3

    $$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

    Oba kierunki łączone są wzorem na długość wektora. Większe wagi sprawiają, że filtr jest nieco bardziej odporny na szumy niż Roberts, lekko wygładzając obraz.

– **operator Prewitta** - w naszej implementacji używamy 8 masek 3×3 (pionowa, pozioma i 6 ukośnych). Dla każdego regionu obliczane są odpowiedzi wszystkich masek, a jako wynik końcowy wybierana jest wartość maksymalna.

Zastosowania wszystkich wymienionych wyżej filtrów znajdują się na Rysunkach 24, 25 i 26.

Rysunek 18: Krzyż Robertsa

Rysunek 19: Operator Sobela

Rysunek 20: Operator Prewitta

3.3 Operacje morfologiczne

Na początek musimy wprowadzić dwa pojęcia używane w operacjach morfoloficznych. Elementem strukturalnym nazywamy mały obraz (zbiór) o dowolnej wielkości i kształcie, w którym piksele o wartości 1 to piksele używane podczas obliczeń. Punkt wyróżniony środek tego lokalnego układu współrzędnych. Użytkownik może wybrać rozmiar elementu strukturalnego (3, 5, 7 lub 9) oraz jego kształt: prostokąt, krzyż, elipsa, linia pozioma lub linia pionowa. Przed operacjami obraz konwertowany jest do skali szarości.

- **erozja** - polega na przypisaniu pikselowi centralnemu minimalnej wartości jasności z obszaru pod elementem strukturalnym.
- **dylatacja** - polega na przypisaniu pikselowi centralnemu maksymalnej wartości jasności z obszaru pod elementem strukturalnym.
- **otwarcie** - polega na wykonaniu erozji i następnie dylatacji. Ta operacja usuwa małe jasne obiekty, rozłącza niektóre elementy i wygładza kontury.

Rysunek 21: Erozja (SE=3, elipsa)

Rysunek 22: Dylatacja (SE=5, krzyż)

Rysunek 23: Otwarcie (SE=3, prostokąt)

- **zamknięcie** - polega na wykonaniu dylatacji i następnie erozji. Ta wypełnia małe dziury, wygładza linię brzegową i łączy blisko leżące elementy.
- **Hit-or-Miss** - ta operacja służy do wykrywania konkretnych wzorców na obrazie. Realizację można podzielić na trzy etapy: wykonanie erozji obrazu oryginalnego z maską wzorca (lokalizacja poszukiwanego kształtu), wykonannie erozji negatywu obrazu z drugą maską (zbadanie tła) i wyznaczenie części wspólnej obu wyników. Piksel wynikowy pojawia się tylko tam, gdzie oba warunki zostały spełnione jednocześnie. Użytkownik wybiera prób binaryzacji i wzorzec: izolowany piksel, kwadrat 2×2, krzyż i dwa rodzaje przekątnych.
- **szkieletyzacja** - operacja iteracyjnie "odchudza" obiekty binarne, aż do uzyskania szkieletu o grubości jednego piksela. Algorytm w każdej iteracji wykonuje operację otwarcia i erozji, wyznacza różnice między nimi, które tworzą szkielet.

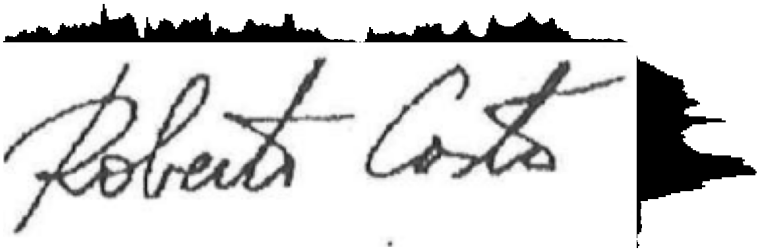
Rysunek 24: Zamknięcie (SE=7, linia pozioma)

Rysunek 25: Hit-or-Miss (przekątna)

Rysunek 26: Szkieletyzacja

3.4 Inne funkcjonalności

- **informacje o obrazie** - w tej sekcji użytkownik może wyświetlić podstawowe informacje o obrazie takie jak: nazwa pliku, ścieżka dostępu, wymiary (px), rozmiar (KB), tryb koloru, głębia bitowa oraz statystyki jasności (minimalna, maksymalna, średnia i odchylenie standardowe). Przykład przedstawia Rysunek 28.
- **projekcja pionowa i pozioma** - narzędzie służące do analizy rozkładu jasności wzdłuż osi obrazu. Algorytm sumuje wartości jasności pikseli wzdłuż osi poziomych (suma wierszy) oraz pionowych (suma kolumn). Przed obliczeniami obraz jest poddawany operacji negatywu i konwersji do skali szarości, aby przygotować dane do sumowania. Projekcja wyświetlana jest w osobnym okinie. Narzędzie to jest przydatne w systemach rozpoznawania pisma oraz przy weryfikacji podpisów. Na Rysunku 27 można zaobserwować wyraźne spadki wartości na wykresie, które odpowiadają pustym przestrzeniom między słowami.



Rysunek 27: Projekcja pionowa i pozioma

Informacje

Plik:	sobel.png
Ścieżka:	C:/Users/Kasia/Desktop/studia/semestr_6/Biometria/Biometria/Project_1/saved_img/sobel.png
Rozmiar:	200 × 300 px
Rozmiar pliku:	114.2 KB
Tryb:	RGB
Głębia:	24-bit
Min jasność:	0
Max jasność:	253
Śr. jasność:	112.3
Odch. std:	63.0

Rysunek 28: Informacje o obrazie

- **histogram** - wyświetlany jest w panelu bocznym. Obsługuje zarówno tryb RGB (3 osobne histogramy) i obraz w skali szarości.
- **wyrównanie histogramu** - głównym zadaniem tej metody jest automatyczna poprawa kontrastu. Algorytm dąży do uzyskania płaskiego histogramu, czyli takiego, w którym każdy poziom jasności występuje z podobną częstotliwością. Konwertuje on obraz na skalę szarości i wykorzystuje dystrybuantę histogramu do stworzenia tabeli mapowania, która rozciąga najczęściej występujące poziomy jasności.

  Na Rysunku 29 i 30 widać odpowiednio wyrównany histogram oraz obraz po wykonaniu tej operacji.

Rysunek 29: Histogram

Rysunek 30: Wyrównanie histogramu

4 Podsumowanie

W ramach projektu stworzyliśmy aplikację do przetwarzania obrazów, w której wszystkie algorytmy zostały zaimplementowane samodzielnie, bez wykorzystania gotowych bibliotek graficznych. Filtry można na siebie dowolnie nakładać – wynik jednej operacji staje się wejściem dla kolejnej, co pozwala na tworzenie złożonych procesów obróbki. Ponadto użytkownik może eksperymentować z różnymi parametrami i na bieżąco obserwować zmiany.

5 Źródła

- `https://pages.mini.pw.edu.pl/~rafalkoj/www/?Dydaktyka:2025%2F2026:-_Biometria`
- `https://brandonrohrer.com/convert_rgb_to_grayscale.html`
- Digital Image Processing, Pearson (2007), Rafael C. Gonzalez, Richard E. Woods
- `http://www.algorytm.org/przetwarzanie-obrazow/filtrowanie-obrazow.html`